COVID-19 CT-Scan Classification — Project Report

AI/ML Engineer Hiring Assignment submission. Problem: binary classification of chest CT-scan images into COVID / non-COVID. All training and evaluation are designed to run on CPU only.Submitted by Amudala Ramyasri

Part 1 — Problem Definition

Problem chosen: Disease classification — binary classification of chest CT-scan images as *COVID-19 positive* or *non-COVID*.

Why it is clinically relevant: During surges in respiratory illness, RT-PCR testing capacity and turnaround time can become a bottleneck. Chest CT imaging is widely available in hospitals and is sensitive to characteristic findings (e.g. ground-glass opacities) that can appear even when PCR results are pending or borderline. An automated triage classifier does not replace PCR or radiologist judgment, but can help flag scans for prioritized review and provide a fast, consistent second signal in resource-constrained settings.

Why this problem was selected: It is a well-scoped binary classification task with a clean, publicly available, moderately sized dataset — large enough to be meaningful, small enough to train and iterate on entirely on CPU within the assignment's time and compute constraints, while still being a realistic, clinically motivated imaging problem rather than a toy task.

Part 2 — Dataset

Property	Value
Source	Kaggle — "SARS-COV-2 Ct-Scan Dataset" (plameneduardo/sarscov2-ctscan-dataset)
Original authors	Soares, E. et al. — SARS-CoV-2 CT-scan dataset
Total images	~2,482 CT-scan images
Patients	120 patients (hospitals in Sao Paulo, Brazil)
Classes	2 — COVID, non-COVID
Class balance	Roughly balanced (~1,252 COVID / ~1,230 non-COVID per the original dataset card)
Format	PNG images, variable resolution, RGB-encoded grayscale CT slices

Challenges within the dataset:

- Single-source data: all images come from one set of hospitals/scanners, so the model may not generalize to other CT equipment, acquisition protocols, or populations.
- Patient-level leakage risk: multiple slices can come from the same patient. Without patient-aware splitting, the same patient's anatomy could appear in both train and test sets, inflating apparent performance. (See Part 3 for how this is handled / its limitation in the current split.)

- Variable image quality and framing — some slices include more of the chest cavity than others, and pre-processing (e.g. windowing) is not standardized across sources.
- No pixel-level (segmentation) labels — only image-level COVID/non-COVID labels, so the model cannot be supervised to localize abnormal regions directly.

Why this dataset was chosen: it is publicly available, ethically sourced and de-identified, and a reasonable size for CPU-only training — large enough to train a meaningful classifier, small enough to iterate on quickly without a GPU.

Part 3 — Model Development

Data preprocessing: Images are read with OpenCV, converted BGR→RGB, resized to 160x160 (a deliberate choice to keep per-image compute low enough for CPU training while retaining enough detail for the classifier), and scaled to [0,1]. MobileNetV2's expected preprocessing (channel-wise scaling/centering) is applied inside the model graph, so the saved model is self-contained for inference.

Data augmentation: Random horizontal flip, small rotation (±5%), small zoom (±10%), and mild contrast jitter — implemented as Keras preprocessing layers so they run as part of the training graph (only during training, not at eval/inference). Augmentation is intentionally light: CT images are not natural photos, so aggressive color/hue jitter or large rotations could distort clinically meaningful structure.

Train/Validation/Test split: Stratified, file-level split — 15% held out as test, then 15% of the remainder held out as validation (net ≈72% train / 12% val / 15% test), seeded for reproducibility (seed=42). The exact split is saved to results/split_indices.json so evaluate.py always scores the same held-out set.

Note: Known limitation: the split is stratified by class but not by patient. Because this dataset's metadata doesn't cleanly expose a patient ID per file in the public release, some patient overlap between splits is possible, which can optimistically bias test metrics. This is flagged explicitly here and addressed as a priority item in Part 5 (Improving the Model).

Model architecture: MobileNetV2 (ImageNet-pretrained) as a feature extractor, with a custom head: GlobalAveragePooling2D → Dense(128, ReLU) → Dropout(0.3) → Dense(1, sigmoid).

Why MobileNetV2: Several ImageNet-pretrained CNN backbones were considered for this task — ResNet50, Xception, DenseNet121/201, MobileNet, and MobileNetV2.

- CPU feasibility: MobileNetV2 was designed for mobile/edge inference — it has far fewer FLOPs and parameters (~3.4M) than deeper backbones like ResNet50 (~25M) or DenseNet201 (~20M), which directly translates to faster training and inference on CPU-only hardware, the hard constraint for this assignment.
- Acceptable accuracy trade-off: lightweight backbones like MobileNetV2 are well documented to trade a few points of top-line accuracy for a large reduction in compute cost compared to deeper networks. Under a CPU-only constraint, that trade-off favors a model that actually finishes training in a reasonable time over one that's marginally more accurate but impractical to iterate on locally.
- Transfer learning is well-justified here: a few thousand images is small for training a CNN from scratch, so ImageNet features (edges, textures, shapes) give a strong starting point that a randomly-initialized network of any size could not match with this little data.

Loss function: Binary cross-entropy — natural choice for a single-sigmoid-output binary classifier with roughly balanced classes.

Optimizer: Adam. **Learning rate:** 0.001 for the head-only phase, dropped to 1e-05 for fine-tuning (with ReduceLROnPlateau on validation loss) — small fine-tuning LR avoids destroying the pretrained features when unfreezing the top of the backbone.

Batch size: 16 — kept small deliberately, since smaller batches keep peak memory and per-step latency manageable on CPU-only hardware.

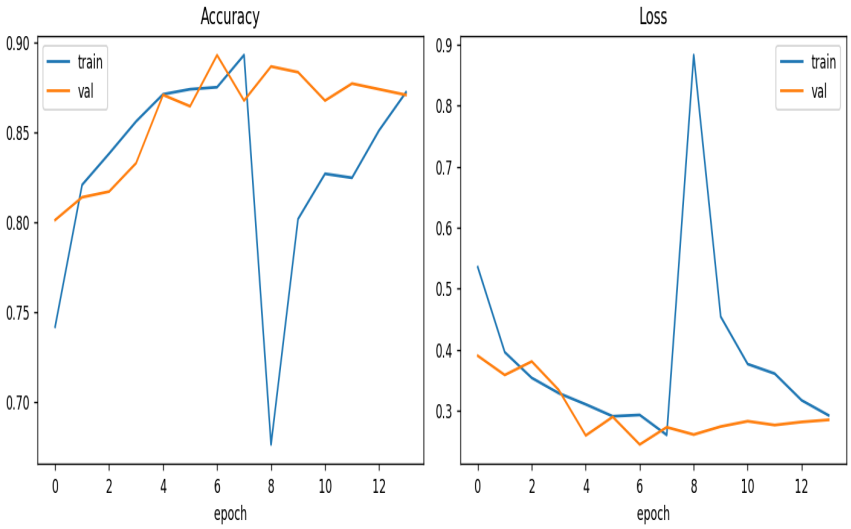
Epochs: up to 15, split into two phases — phase 1 (epochs 1–8): backbone frozen, only the new head trains; phase 2 (remaining epochs): top ~30 layers of the backbone are unfrozen and fine-tuned. EarlyStopping (patience 5, restores best weights) and ModelCheckpoint (best val_loss) prevent overfitting and wasted CPU time.

Part 4 — Evaluation

Results below are from this project's own trained model (results/metrics.json), evaluated on the held-out test split.

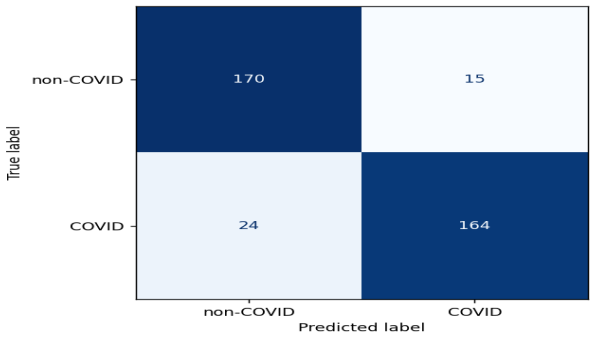
Metric	Value
accuracy	0.8954
precision	0.9162
recall	0.8723
f1	0.8937
roc_auc	0.9651

Training & validation curves

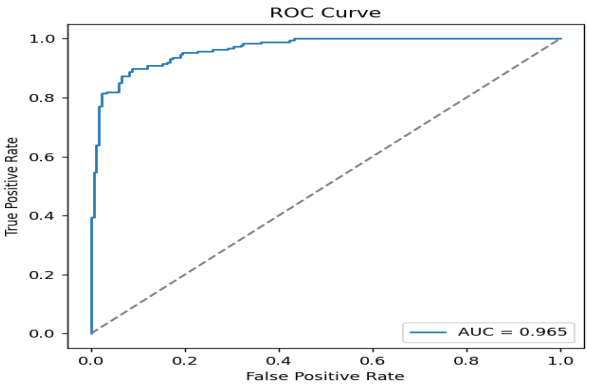


Training/validation accuracy and loss across both training phases.

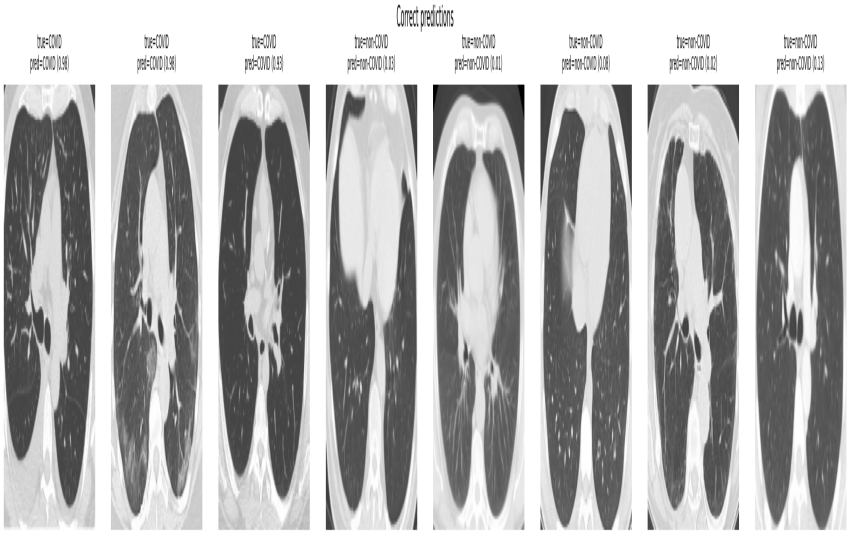
Confusion matrix



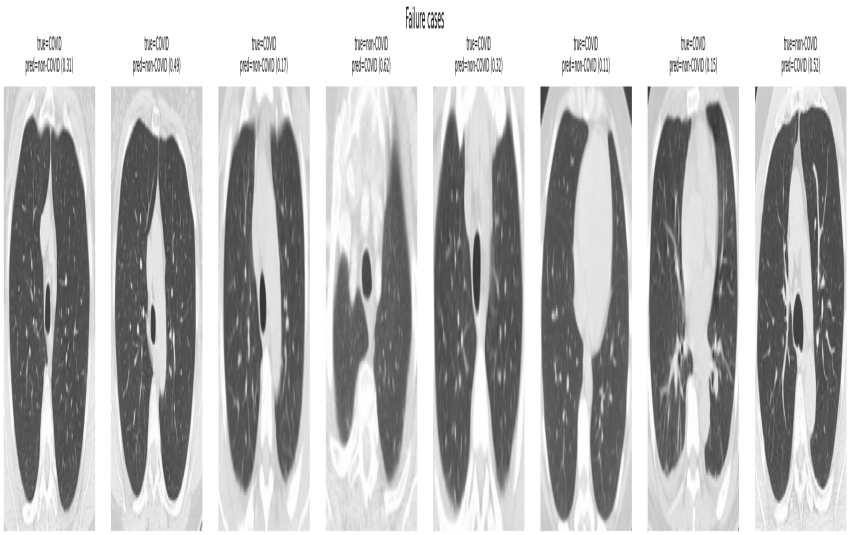
ROC curve



Sample predictions



Correctly classified examples.



Misclassified examples (failure cases).

Discussion

Where the model performs well (expected): Cases with clear, classic ground-glass or consolidation patterns typical of the training distribution, and images framed/cropped similarly to the bulk of the training data.

Where the model is expected to fail: Borderline/early-stage cases with subtle findings; images with atypical framing, noise, or artifacts not well represented in training; non-COVID cases with other lung pathology that visually resembles COVID findings (a known confusion in this literature).

Possible reasons for failure: Limited dataset diversity (single source), image-level labels without abnormality localization (the model can't "point at" the relevant region, making subtle cases harder to fit), and possible patient-level split leakage masking some generalization weaknesses until tested on a truly external patient/hospital.

Part 5 — Improving the Model

If this were to become part of a real-world medical imaging product, priorities in roughly the order I'd tackle them:

Patient-aware, multi-site evaluation. Re-split (and ideally re-collect) data so no patient appears in both train and test, and add an external test set from a different hospital/scanner. **Benefit:** honest estimate of real-world generalization. **Trade-off:** requires more data-sourcing effort and may reveal a real accuracy drop versus the single-source numbers reported here.

Better preprocessing. CT-specific windowing/normalization (e.g. lung-window leveling) instead of generic RGB scaling. **Benefit:** emphasizes clinically relevant contrast. **Trade-off:** needs DICOM-level metadata, which this PNG-only public dataset doesn't provide — would require sourcing a DICOM dataset.

Transfer learning from a domain-specific foundation model. Use a chest-CT or medical-imaging pretrained backbone (e.g. CT-CLIP-style or RadImageNet weights) instead of generic ImageNet weights. **Benefit:** features already tuned to medical imaging statistics, likely better small-data performance. **Trade-off:** larger/heavier checkpoints, license/availability constraints, possibly GPU-only inference, eroding the CPU-only property of this build.

Self-supervised / semi-supervised pretraining on unlabeled CT scans. Pretrain on a larger pool of unlabeled chest CTs before fine-tuning on labeled COVID data. **Benefit:** reduces dependence on scarce labels. **Trade-off:** meaningfully more compute and engineering complexity than transfer learning from existing weights.

Better / more diverse data collection. Actively source data across more hospitals, scanners, and patient demographics. **Benefit:** the single highest-leverage way to improve real-world robustness. **Trade-off:** slow, costly, and subject to data-sharing/privacy agreements.

Calibration. Apply temperature scaling or Platt scaling on a held-out calibration set so predicted probabilities reflect true likelihoods. **Benefit:** a 0.51 vs 0.93 COVID probability becomes meaningfully different to a clinician. **Trade-off:** needs a dedicated calibration split, and calibration can drift if the deployment population shifts from the calibration population.

Hyperparameter optimization. Systematic search (e.g. small Bayesian or grid search) over learning rate, fine-tune depth, dropout, and augmentation strength. **Benefit:** likely a few more points of accuracy/F1 for free. **Trade-off:** multiplies compute cost, which competes directly with the CPU-only constraint.

Ensembling. Combine 2–3 lightweight backbones (e.g. MobileNetV2 + a small EfficientNet) by averaging predictions. **Benefit:** typically a reliable, low-risk accuracy boost. **Trade-off:** multiplies inference cost and model maintenance burden for a relatively modest gain.

Localization / explainability (e.g. Grad-CAM). Even without segmentation labels, Grad-CAM-style saliency maps can show *where* the model is focusing. **Benefit:** critical for radiologist trust and for catching the model focusing on spurious artifacts instead of anatomy. **Trade-off:** visualization only — doesn't directly improve accuracy, and saliency maps can be unreliable/misleading if over-interpreted.

Active learning. Once deployed (in a research/triage-support capacity), route low-confidence predictions to radiologists and feed their corrections back into retraining. **Benefit:** targets labeling effort where it matters most. **Trade-off:** needs a feedback/labeling pipeline and careful governance around using clinician corrections as training data.

Part 6 — AI Assistant

A lightweight project assistant lives in `assistant/` and is exposed via CLI (`python assistant/assistant.py`).

Design: `assistant/knowledge.py` compiles a single "project facts" context block — problem statement, dataset summary, model rationale, training configuration, limitations, improvement plan, deployment guidance, and (live) the contents of results/metrics.json. `assistant/assistant.py` answers each question through a layered fallback chain, tried in order:

- **Guaranteed-match layer:** the assignment's six required questions (what problem it solves, which dataset was used, why this model was selected, how it was trained, what the limitations are, how to interpret the results) — and close rewordings of them — are matched directly against the project-facts dictionary, so these always return a correct, complete answer regardless of which backend below is available.
- **Local open-source LLM via Ollama** (free, offline, no API key): for anything outside the six required questions, the same context block is passed as a system prompt to a small local model (e.g. llama3.2:1b), letting it reason and respond more naturally instead of relying on fixed keyword matches.
- **Rule-based keyword fallback:** if Ollama isn't running, a broader keyword router still answers from the same project-facts dictionary, so the assistant works out-of-the-box with zero setup, zero cost, and no internet dependency.

It can answer the questions the assignment specifies: what problem the model solves, which dataset was used, why this model was selected, how it was trained, what its limitations are, and how to interpret its results — all grounded in the same facts documented in this report, so the assistant's answers stay consistent with the written report and with results/metrics.json as it gets updated.

Note: This is intentionally lightweight, per the assignment brief — a context-grounded Q&A; layer over the project's own facts, not a production agent with tool use, memory, or multi-turn planning.